



TECHNICAL WHITEPAPER

One repo in. Three truths out.

A three-check clinic for one codebase

TriRepo checks any Node/Express codebase for the three problems that quietly cost engineering teams hours: documentation that lies about the code, crashes nobody can reproduce, and dependency upgrades that break more than release notes admit. Every check runs twice — once instantly and deterministically in the visitor's browser, and once as a deep, evidence-backed pass by IBM Bob 2.0 on a purpose-built sample application.

Built with IBM Bob 2.0

lablab.ai hackathon entry

MIT licensed

LIVE DEMO

`tri-repo-ui-`
`production.up.railway.app`

SOURCE

`github.com/TeevinsCrypt/TriRepo`

VERSION

September
2026

01 The problem

Three moments quietly cost engineering teams the most time, and they are rarely treated as one problem. A README documents a script, a route, or a Node version that no longer matches the code, and a new contributor loses an afternoon to it. A stack trace arrives in a bug report or a log, and reproducing it means manually tracing frames across a codebase nobody currently has open. A dependency bump looks safe in the changelog, but a handful of call sites use APIs the new major version silently changed or removed.

Each of these looks like a narrow, tool-shaped problem — a linter for docs, a stack-trace viewer, a dependency auditor. TriRepo's premise is that they are the same failure mode wearing three faces: the repository's stated understanding of itself has drifted from what the code actually does. Treating them as one product, run against the same repo, on the same day, is more useful than three separate tools that never talk to each other.

02 Approach: two layers, one repo

Every check in TriRepo has two layers that answer the same question at different depths:

Live

Runs in the visitor's browser against **any public GitHub repository**, in seconds. Deterministic, rule-based, no AI, no login, nothing stored.

Deep pass

IBM Bob 2.0 running as an autonomous coding agent against `patient/`, a purpose-built sample application, producing a written report with evidence: file paths, line numbers, failing tests, reproduced errors.

The live layer proves the idea scales to arbitrary code the moment a judge or user pastes a URL. The deep-pass layer proves what a capable coding agent can actually verify when it is allowed to plan, edit, and test — not just pattern-match. Neither layer is a mockup of the other; both run for real, against real code, on every visit.

Why a sample app, not a random public repo, for the deep pass? Bob's findings need to be checkable against ground truth. `patient/` is a small but real Express 4 application (Vaultline, an invoice API) with problems planted on purpose and recorded in an answer key written only after all three passes were complete, so Bob could not have read ahead. Every number in this document that describes Bob's deep pass traces back to that answer key and to the reports Bob wrote.

03 The three checks

3.1 LIES — docs vs. code

Live: fetches a repository's README, CONTRIBUTING.md, and package.json directly from GitHub, then flags documented npm run scripts that do not exist, a documented Node version that does not match the engines field, and a documented port that does not match the code's configured default.

IBM Bob's deep pass: two subagents ran in parallel with no shared state — one read only documentation files, the other read only source and config — and produced two independent claim lists. Bob then diffed the two lists in Plan mode and found **11 contradictions**, including two real CHANGELOG inconsistencies that had never been planted. It applied the three safest fixes directly to the docs and documented the remaining eight with its reasoning.

Example finding: the README documents every API route under /api/v2/...; the server mounts them all under /api/v1/.... A new contributor following the README would call routes that do not exist.

3.2 CRASH — stack trace to fix

Live: a visitor pastes any Node, Python, Java, Go, or Ruby stack trace. TriRepo parses the frames, separates the visitor's code from dependency and runtime frames, resolves each file against the target repository (or against GitHub's tree API when a repo is given), and renders the source at the failing line with a plain-language, pattern-based hint about the likely cause.

IBM Bob's deep pass: starting from a single captured stack trace (patient/CRASH.txt) and nothing else, Bob traced the failure to dateService.js:39, where an undefined payment-terms value reaches .toLowerCase(). It followed the call chain through two more files unprompted, wrote a unit test and an integration test that reproduced the crash (confirmed RED), added a minimal guard that returns a structured 400 VALIDATION_ERROR, confirmed both new tests passed (GREEN), and re-ran the full suite at 35 of 35 passing.

3.3 BUMP — upgrade blast radius

Live: reads a repository's package.json, following npm workspaces, compares every declared dependency against the latest version published to the npm registry, and flags major-version gaps — the version boundary where breaking changes are most likely to live. A **Find usages** action then greps the repository for every import and direct use of a flagged package's binding.

IBM Bob's deep pass: an explore subagent inventoried roughly 60 Express call sites across patient/src. Bob then built a disposable sandbox project, installed Express 5.2.1 for real, and issued live HTTP requests against it to confirm or refute each suspected break — no assumptions from release notes alone.

EXPRESS 4 API	BEHAVIOUR ON EXPRESS 5	BOB'S OUTCOME
<code>app.del()</code>	TypeError at startup	Patched
<code>app.use('*', ...)</code>	PathError at startup	Patched
<code>req.param()</code>	TypeError on every GET-by-id	Patched
<code>res.json(obj, 201)</code>	Silently returns 200 instead of 201	Documented, left unpatched

All four suspected breaks reproduced with zero false positives. Three were patched so the code now works on both Express 4 and 5; the fourth was left as a documented residual risk, with the reasoning recorded in `patient/UPGRADE.md` rather than silently fixed, because a silent status-code change is easy to miss in review.

04 System architecture

TriRepo is a single Node.js/Express server with no database and no user accounts. It has two responsibilities: serve the static landing page and clinic UI, and forward a narrow, allow-listed set of outbound requests when a visitor's browser cannot reach GitHub or npm directly (CORS or rate-limit fallback).

ROUTE	PURPOSE
/	The landing page
/app	The clinic: three tabs, live tools plus Bob's deep-pass reports
/api/proxy	Forwards to <code>raw.githubusercontent.com</code> and <code>registry.npmjs.org</code> only, over HTTPS, used only on fetch failure
/api/sample-trace	Serves the captured crash trace used by the Crash tool's demo sample

The live checks execute entirely client-side in `app/public/live-tools.js`: stack-trace parsing (Node/Python/Java/Go/Ruby frame grammars), file resolution against GitHub's tree API with a raw-content probing fallback when the API is rate-limited or unreachable, dependency-version comparison against the npm registry using semver rules, and a static-analysis pass that follows ES module and CommonJS import bindings to find usages. No step calls a language model; every result is reproducible from the same inputs.

```
GET https://api.github.com/repos/<owner>/<repo>/git/trees/main?recursive=1
```

```
→ file tree, used to resolve stack-trace frames to real file paths
```

```
GET https://registry.npmjs.org/<package>/latest
```

```
→ latest published version, compared against the declared semver range
```

Resilience

GitHub's unauthenticated API allows 60 requests per hour per visitor. When that limit is hit, or the API is otherwise unreachable, Crash falls back to parallel raw-content probing of candidate paths, and Bump disables Find usages with an explicit on-screen note rather than failing silently or showing stale data.

05 The patient application

`patient/` is **Vaultline**, a small but functioning Express 4 invoice-management API with 35 passing tests. It exists purely as ground truth for Bob's deep pass and carries three categories of problem, planted on purpose:

8

documentation contradictions across
README, CONTRIBUTING, and
CHANGELOG

1

null-dereference crash, with a real
captured stack trace

4

Express 4-only API uses that break on
Express 5

`patient/PLANTED.md` is the answer key. It documents not only what was planted but what Bob found that nobody planted — two real CHANGELOG errors, for instance — a distinction that matters for evaluating whether Bob's findings are genuine analysis or pattern-matching against a fixture.

06 How IBM Bob 2.0 was used

Each check ran as its own Bob task in the Bob IDE, started fresh with no memory of what earlier passes had planted or found — a deliberate constraint so later passes could not read ahead. Prompts are saved and re-runnable in prompts/.

CAPABILITY	USED IN	WHAT IT DID
Plan mode	Every pass	Designed the app, the contradiction diff, the fix, and the patch order
Ask mode	Passes 1–3	Read-only analysis before any edit was made
Agent mode	Every pass	Every file write and code edit
Parallel subagents	Pass 1 — Lies	Docs and code read independently, then compared
Explore subagent	Pass 3 — Bump	Catalogued ~60 Express call sites without loading all source into the parent context
Full-repo context	Passes 2–3	Followed the crash through three files, and verified break patterns across the src tree, unprompted
Test-first debugging	Pass 2 — Crash	Wrote a failing test from the trace before touching source: RED, then GREEN
Sandboxed reproduction	Pass 3 — Bump	Installed real Express 5.2.1 and issued real HTTP requests to prove each break

TriRepo does not use watsonx.ai or watsonx Orchestrate. The live checks run on fixed rules with no model in the loop at runtime; all model-driven work in the project was IBM Bob 2.0, during the three build passes described above.

07 Results, in numbers

11

documentation contradictions found

1

crash fixed, RED to GREEN

4

Express 5 breaks reproduced, 0 false positives

35/35

tests passing after every fix

08 Authorship & disclosure

IBM Bob 2.0 produced the sample application, its tests and every planted problem, all three deep-pass reports, every fix and patch inside `patient/`, the original prompts and docs, and the first version of the UI, including the live Lies check. After the project's Bob usage allowance was exhausted, the live Crash and Bump tools, the landing page, and the current visual design (`app/public/`) were built with Claude Code. This is recorded file by file in the repository's `HACKATHON.md`. Nothing in `reports/` or `patient/` was touched after Bob's passes.

09 Limitations

- **Public repositories only.** Lies and Bump need an npm project with a `package.json` at the repository root; Bump follows npm workspaces but not other monorepo tools.
- **Unauthenticated GitHub API limits.** 60 requests per hour per visitor; Crash and Bump degrade gracefully (raw-content probing, Find usages disabled) rather than fail silently.
- **Bump reasons about version ranges, not API surfaces.** It flags major-version gaps using semver; it does not know which specific APIs changed. Find usages narrows the search to real call sites, scanning up to 300 source files.
- **Crash's live layer is heuristic.** Its pattern rules point at the failing line and a likely cause; they do not constitute proof of root cause. That is what Bob's deep pass demonstrates is possible with a full read-write-test loop.

10 Roadmap

DIRECTION	WHAT IT ADDS
Bob's deep pass on any repo	Run the same three-pass methodology against a visitor's own repository, not only the bundled sample
Authenticated GitHub sign-in	Private repositories and materially higher API limits
Ecosystems beyond npm	Bump support for pip, Go modules, and Maven
API-surface diffing	Map a dependency's release notes to a repository's actual call sites, the way Bob did by hand for Express 5

Deliberately out of scope for now: user accounts, a persistence layer, and an automatic "fix everything" action. Bob's own deep pass fixes only what it judges safe and documents the rest with its reasoning — TriRepo's live tools follow the same discipline.

11 Technology & access

LAYER	TECHNOLOGY
Server	Node.js, Express 4

Client	Vanilla HTML/CSS/JavaScript, no framework
Sample app tests	Vitest
Report rendering	marked (Markdown → HTML at request time, no build step)
Data sources	GitHub REST/tree API, npm registry API
Hosting	Railway, deployed from the main branch
License	MIT

Try it. Live demo: trirepo-ui-production.up.railway.app · Source, reports, prompts, and all 13 IBM Bob session screenshots: github.com/TeevinsCrypt/TriRepo